

N-Body Simulation: Milestone 1

Hafsa Tanveer
IBA, SMCS
29041

h.tanveer.29041@khi.iba.edu.pk

Manal Fatima
IBA, SMCS
29018

m.fatima.29018@khi.iba.edu.pk

Areeba Salman
IBA, SMCS
29075

a.salman.29075@khi.iba.edu.pk

Abstract—This paper explores naive N-Body simulation using the naive N-body algorithm with Leapfrog integration of ordinary differential equations, and then further makes it efficient with the Barnes-Hut algorithm.

Keywords—N-body, Leapfrog, Euler, Barnes Hut

I. INTRODUCTION

The N-body problem calculates how n number of bodies interact in a system, which is commonly used in various science fields such as astrophysics, plasma physics, and fluid dynamics. We are implementing the N-body simulation firstly using the naive algorithm. The naive algorithm can be implemented via two different methods, Leapfrog and Euler. We will discuss the differences and why we chose to go ahead with Leapfrog. We will then further improve the time efficiency of the naive algorithm using the Barnes-Hut algorithm while exploring octrees and quadrees.

II. EULER VS LEAPFROG

A. Euler Integration

Euler integration is a first-order numerical method where the next state is calculated directly from the current state:

$$\begin{aligned}x_{\{n+1\}} &= x_n + h v_n \\v_{\{n+1\}} &= v_n + h a_n \\y_{\{n+1\}} &= y_n + h f(t_n, y_n)\end{aligned}$$

Euler integration is computationally simple but is not symplectic, meaning it does not conserve system energy. Over time, numerical error accumulates and causes unstable orbital behaviour where bodies gain artificial energy and drift from expected trajectories.

Euler is also numerically unstable for stiff equations, where numerical solutions grow rapidly even when true solutions remain bounded.

B. Leapfrog Integration

Leapfrog integration is a second-order symplectic method that evaluates position and velocity at staggered time steps:

In practice, it is implemented using the Kick-Drift-Kick formulation:

$$\begin{aligned}v_{t+\frac{1}{2}} &= v_t + \frac{a_t(dt)}{2} \\x_{t+1} &= x_t + v_{t+\frac{1}{2}}dt \\v_{t+1} &= v_{t+\frac{1}{2}} + \frac{a_{t+1}(dt)}{2}\end{aligned}$$

This sequence consists of an initial half-step velocity update (Kick), a full-step position update (Drift), and a final half-step velocity update (Kick).

Leapfrog preserves energy and angular momentum and is stable for oscillatory motion, making it highly suitable for physical simulation.

C. Why Leapfrog Was Chosen

Leapfrog integration was selected over Euler integration because it provides higher numerical accuracy and physical stability while maintaining similar computational cost. Leapfrog is a second-order method, whereas Euler is first order, meaning Leapfrog produces significantly lower accumulated error over time. Despite this improvement, both methods require approximately the same number of function evaluations per step.

Leapfrog is particularly suitable for physical simulations because it is symplectic, meaning it preserves energy and angular momentum in Newtonian systems. It is also time-reversible and stable for oscillatory motion, which is critical in orbital and gravitational simulations. In contrast, Euler integration introduces artificial energy drift, causing orbits to diverge over time. For long-duration N-body simulations, this instability makes Euler unsuitable, which is why Leapfrog was chosen for this implementation.

III. NAÏVE N-BODY ALGORITHM AND SOFTENING

A. Time Complexity

The time complexity for the naive N-body algorithm is $O(n^2)$ as it simulates the interaction of each object with another. For a system of n bodies, calculating the net force on a single body requires iterating through the other $n-1$ bodies, and this process is repeated n times. The quadratic time complexity is a problem when dealing with a large number of bodies. Doubling the number of particles quadruples the computational load, making this approach unfeasible for large-scale simulations without optimization.

B. Gravitational Softening

A critical component of the force calculation is the softening parameter. In a discrete time-step simulation, two particles may approach each other so closely that the distance approaches zero, causing the force to become arbitrarily large. To mitigate this numerical explosion, we modified the distance calculation to include a softening term.

$$r_{\{soft\}} = \sqrt{\{r^2 + \epsilon^2\}}$$

(where r is replaced in the formula for gravitational force)

This term effectively smooths the gravitational potential at short ranges, preventing division-by-zero errors while maintaining accuracy at larger distances.

IV. BARNES HUT ALGORITHM

The Barnes-Hut algorithm reduces the computational complexity of N-body force calculations from $O(n^2)$ to $O(n)$

$\log n$).by grouping distant bodies and approximating them as a single virtual body located at their combined center of mass. This significantly reduces the number of pairwise force calculations required while maintaining acceptable accuracy for large-scale simulations.

A. Octree Spatial Partitioning

This optimization is achieved using a hierarchical tree data structure, specifically an octree in three-dimensional space. The root node represents the entire simulation space, and each internal node recursively subdivides space into eight octants until each subdivision contains at most one body. Internal nodes store aggregate properties such as total mass and center of mass, while leaf nodes store individual bodies or remain empty. This hierarchical spatial partitioning enables efficient approximation of distant particle groups.

B. Force Calculation and Tree Traversal

During force computation, the tree is traversed starting from the root node. For each node, a decision is made whether the group of bodies can be approximated as a single virtual body. This decision is based on the ratio s/d where s is the size of the cell and d is the distance between the current body and the node's center of mass. If this ratio is less than a threshold parameter θ the grouped approximation is used; otherwise, the traversal continues into the node's children to compute forces at finer resolution.

C. Overall Speedup

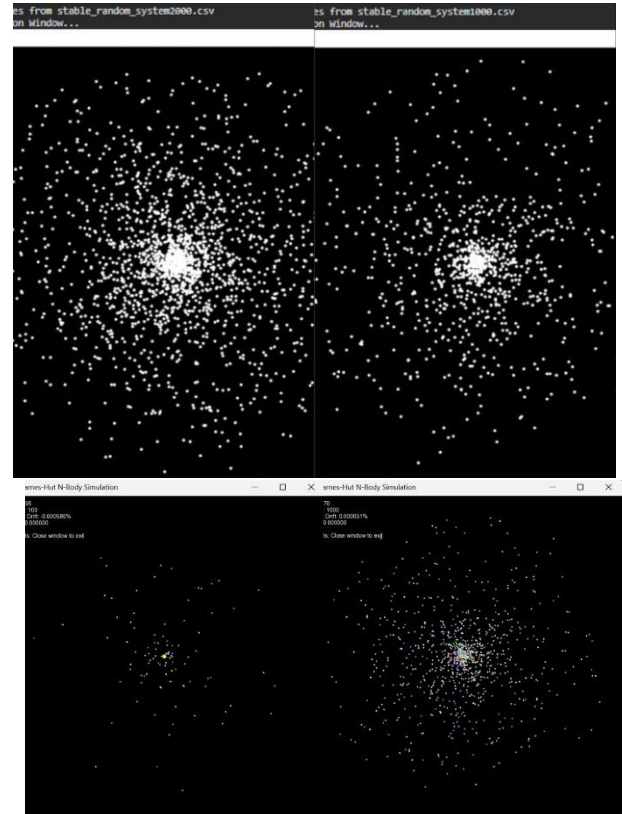
Because distant clusters are treated as single interactions rather than multiple pairwise interactions, the total number of force calculations is drastically reduced. Since tree construction and traversal scale logarithmically with the number of bodies, the overall computational complexity approaches $O(n \log n)$, making Barnes–Hut suitable for large N-body simulations that would otherwise be computationally infeasible using the naive approach.

V. CHALLENGES FACED

A hardcoded data range in the sample dataset made it difficult as different datasets had varying ranges, so we had to create boundaries by finding the min and max and then padding it by 1%. Another issue was that each node had to have 1 body. Initially, the center of mass was being calculated before subdivision, which we fixed by calculating mass outside the loop. Furthermore, ensuring the correct order of operations for the Kick-Drift-Kick loop was a primary challenge; it is crucial that the force calculation occurs after the position update but before the second velocity update. We also had to find a sweet spot for the value of softening in the naive body implementation.

VI. VISUALISATION

TOP DIAGRAM SHOWS VISUALISATION USING NAÏVE ALGORITHM



BOTTOM DIAGRAM SHOWS IT VIA BARNES HUT

VII. ACKNOWLEDGMENTS

We thank Sir Salman Zaffar for his invaluable guidance and expertise, and our teaching assistant Hamza for his thankless help and resources.

VIII. REFERENCES

- "Euler method," Wikipedia. Available: https://en.wikipedia.org/wiki/Euler_method.
- "Leapfrog integration," Wikipedia. Available: https://en.wikipedia.org/wiki/Leapfrog_integration.
- P. Young, "Leapfrog method and other symplectic algorithms for integrating Newton's laws of motion," UC San Diego. Available: <https://courses.physics.ucsd.edu/2019/Winter/physics141/Assignments/leapfrog.pdf>.
- "Barnes-Hut simulation," Wikipedia. Available: https://en.wikipedia.org/wiki/Barnes%E2%80%93Hut_simulation.
- "A hierarchical $O(N \log N)$ force-calculation algorithm," ACM Digital Library. Available: <https://dl.acm.org/doi/pdf/10.1145/3624062.3624604>.
- "Barnes-Hut Algorithm," YouTube. Available: https://youtu.be/nZHjD3cI-EU?si=NGTQsXyib_5DQjt1.